

1. Write a program to find the determinant of a given square matrix.

Program:

```
#include <stdio.h>
#define N 3
void getCofactor(int mat[N][N], int temp[N][N],
                 int p, int q, int n)
{
    int i = 0, j = 0;
    for (int row = 0; row < n; row++)
    {
        for (int col = 0; col < n; col++)
        {
            if (row != p && col != q)
            {
                temp[i][j++] = mat[row][col];
                if (j == n - 1)
                {
                    j = 0;
                    i++;
                }
            }
        }
    }
}

int determinantOfMatrix(int mat[N][N], int n)
{
    int D = 0;
    if (n == 1)
        return mat[0][0];
    int temp[N][N];
    int sign = 1;
    for (int f = 0; f < n; f++)
    {
        getCofactor(mat, temp, 0, f, n);
        D += sign * mat[0][f]
            * determinantOfMatrix(temp, n - 1);

        sign = -sign;
    }

    return D;
}
```

```

void display(int mat[N][N],
             int row, int col)
{
    for (int i = 0; i < row; i++)
    {
        for (int j = 0; j < col; j++)
            printf(" %d", mat[i][j]);
        printf("\n");
    }
}

int main()
{
    int mat[N][N];
    printf("\nEnter %d x %d matrix:\n", N, N);
    for(int i=0; i<N; i++)
    {
        for(int j=0; j<N; j++)
        {
            scanf("%d", &mat[i][j]);
        }
    }
    printf("Determinant of the matrix is : %d",
           determinantOfMatrix(mat, N));
    return 0;
}

```

Output:

```

Enter 3 X 3 matrix:
2 1 3
1 2 1
3 1 2
Determinant of the matrix is : -8

```

2. Write a program to find a real root of the following equation: $3x - \cos x - 1 = 0$ using Newton-Raphson method, correct up to four decimal places.

Program:

```
#include<stdio.h>
#include<math.h>
double f(double x){return 3*x-cos(x)-1;
}
double df(double x){return 3+sin(x);
}
int main()
{   int i;
    double x0=0.5,x1;
    double eps=1e-6;
    int maxIter=100;
    for(i=0;i<maxIter;i++)
    {
        x1 = x0 - f(x0)/df(x0);
        if(fabs(x1-x0)<eps) break;
        x0 = x1;
    }
    printf("\nRoot = %.6f (%d iterations)\n.",x1,i+1);
    return 0;
}
```

Output:

```
Root = 0.607102 (3 iterations)
```

3. Write a program to solve the following system of equations using Gauss-Seidel iteration method.

$$20x + y - 2z = 17$$

$$3x + 20y - z = -18$$

$$2x - 3y + 20z = 25$$

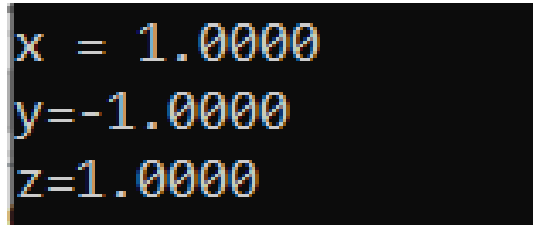
Program:

```
#include<stdio.h>
#include<math.h>
int main()
{
    float x=0,y=0,z=0;
    float x_new,y_new,z_new;
    float error;

    do{
        x_new = (17-y+2*z)/20;
        y_new = (-18-3*x_new+z)/20;
        z_new = (25-2*x_new+3*y_new)/20;
        error = fabs(x_new-x)+fabs(y_new-y)+fabs(z_new-z);

        x=x_new;
        y=y_new;
        z=z_new;
    }while(error>0.0001);
    printf("\nx = %.4f\ny=%.4f\nz=%.4f",x,y,z);
    return 0;
}
```

Output:



```
x = 1.0000
y=-1.0000
z=1.0000
```

4. Write a program to find the approximate value of the root of the equation

$$x^3 - 5x + 1 = 0$$

lying between 0 and 1 by Regula-Falsi method, correct up to four decimal places.

Program:

```
#include<stdio.h>
#include<math.h>
float f(float x){
    return x*x*x - 5*x + 1;
}
int main()
{
    float a = 0, b = 1, c;
    float error;
    do{
        c = (a*f(b)-b*f(a))/(f(b)-f(a));
        if (f(a)*f(c)<0)
            b=c;
        else
            a=c;
        error = fabs(f(c));

    }while(error>0.0001);
    printf("\nApproximate Root=%.4f",c);
    return 0;
}
```

Output:

Approximate Root=0.2017

5. Find a positive root of the following equation using Bisection method correct up to three decimal places:

$$x^3 - x - 1 = 0$$

Program:

```
#include<stdio.h>
#include<math.h>
double f(double x){
    return x*x*x - x - 1;
}
int main()
{
    double a,b,c;
    int maxIter = 100;
    double eps = 1e-6;
    a = 1, b = 2;
    if(f(a)*f(b)>=0)
    {
        printf("\nInvalid initial guesses\n");
        return 1;
    }
    for (int i=0;i<maxIter;i++)
    {
        c = (a+b)/2;
        if(f(a)*f(c)<0)
            b = c;
        else
            a = c;
    }
    printf("\nRoot = %.3f",c);
    return 0;
}
```

Output:

```
Root = 1.325
```

6. Write a program to find the value of $\cos 12^\circ$ from the following table:

x	0	10	20	30	40
cos(x)	1.0000	0.9848	0.9397	0.8660	0.7660

Use Newton's forward difference interpolation formula.

Program:

```
#include<stdio.h>
#include<math.h>
#define N 20
int main()
{
    float h,f,p,d,s;
    float x[N],y[N];
    int i,j,n;
    printf("\nEnter the value of n(number of terms you want to enter):");
    scanf("%d",&n);
    printf("\nEnter the elements of x:\n");
    for(i=1;i<=n;i++)
    {
        scanf("%f",&x[i]);
    }
    printf("\nEnter the elements of y:\n");
    for(i=1;i<=n;i++)
    {
        scanf("%f",&y[i]);
    }

    h=x[2]-x[1];
    printf("\nPlease enter the value of x for which you want to print y:\n");
    scanf("%f",&f);
    p=1;
    d=y[1];
    s=(f-x[1])/h;
    for(i=1;i<=n-1;i++)
    {
        for(j=1;j<=(n-i);j++)
        {
            y[j]=y[j+1]-y[j];
        }
        p=p*(s-i+1)/i;
        d=d+p*y[1];
    }
    printf("\nFor the value of x(%f) the value of y is %0.4f",f,d);
    return 0;
}
```

Output:

```
Enter the value of n(number of terms you want to enter):5
```

```
Enter the elements of x:
```

```
0  
10  
20  
30  
40
```

```
Enter the elements of y:
```

```
1.0000  
0.9848  
0.9397  
0.8660  
0.7660
```

```
Please enter the value of x for which you want to print y:
```

```
12
```

```
For the value of x(12.000000) the value of y is 0.9781
```

7. The following table gives the population of a town (in thousands) during the census years:

Year (x)	1891	1901	1911	1921	1931
Population (y)	46	66	81	93	101

Write a C program to estimate the population in the year 1925 using Newton's Backward Interpolation formula.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
float u_cal(float u, int n)
{
    float temp = u;
    for (int i = 1; i < n; i++)
        temp = temp * (u + i);
    return temp;
}
int fact(int n)
{
    if (n==0 || n==1)
        return 1;
    else
        return n*fact(n-1);
}

int main()
{
    int n=0;
    printf("Please Enter the number of values: ");
    scanf("%d",&n);
    float x[n],y[n][n];
    printf("Please enter the values of x:\n");
    for(int i=0;i<n;i++){
        scanf("%f",&x[i]);
    }
    printf("Please enter the values of y:\n");
    for(int i=0;i<n;i++){
        scanf("%f",&y[i][0]);
    }

    for (int i = 1; i < n; i++) {
        for (int j = n - 1; j >= i; j--)
            y[j][i] = y[j][i - 1] - y[j - 1][i - 1];
    }
```

```

printf("Displaying the backward difference table.....\n\n");
for (int i = 0; i < n; i++) {
    printf("%0.2f\t\t",x[i]);
    for (int j = 0; j <= i; j++)
        printf("%0.2f\t\t",y[i][j]);

    printf("\n");
}

float value = 0;
printf("Please enter the value of f\n");
scanf("%f",&value);

float sum = y[n - 1][0];
float u = (value - x[n - 1]) / (x[1] - x[0]);
for (int i = 1; i < n; i++) {
    sum = sum + (u_cal(u, i) * y[n - 1][i]) /
        fact(i);
}

printf("Value at %0.4f is %0.4f\n",value,sum);
return 0;
}

```

Output:

```

Please Enter the number of values: 5
Please enter the values of x:
1891
1901
1911
1921
1931
Please enter the values of y:
46
66
81
93
101
Displaying the backward difference table.....

1891.00      46.00
1901.00      66.00      20.00
1911.00      81.00      15.00      -5.00
1921.00      93.00      12.00      -3.00      2.00
1931.00     101.00      8.00      -4.00      -1.00      -3.00
Please enter the value of f
1925
Value at 1925.0000 is 96.8368

```

8. Write a program to determine the value of $Y(10)$ using Lagrange interpolation, from the unequally spaced data given below:

X	5	6	9	11
Y	12	13	14	16

Program:

```
#include <stdio.h>

int main() {
    double x[] = {5, 6, 9, 11};
    double y[] = {12, 13, 14, 16};
    int n = 4;
    double xp = 10, yp = 0;

    for (int i = 0; i < n; i++) {
        double term = y[i];
        for (int j = 0; j < n; j++) {
            if (j != i)
                term *= (xp - x[j]) / (x[i] - x[j]);
        }
        yp += term;
    }

    printf("Interpolated value at x = %.2f is y = %.4f\n", xp, yp);
    return 0;
}
```

Output:

```
Interpolated value at x = 10.00 is y = 14.6667
```

9. Write a program to evaluate the following:

$$\int_0^6 \frac{1}{1+x^2} dx$$

Use Simpson's 3/8 rule with 6 intervals.

Program:

```
#include <stdio.h>
#include <math.h>

double f(double x) { return 1.0 / (1.0 + x*x); }

int main() {
    double a = 0.0, b = 6.0;
    int n = 6;
    double h = (b - a) / n;
    double sum = f(a) + f(b);

    for (int i = 1; i < n; i++) {
        if (i % 3 == 0)
            sum += 2.0 * f(a + i * h);
        else
            sum += 3.0 * f(a + i * h);
    }

    double result = (3.0 * h / 8) * sum;

    printf("Integral = %.6f\n", result);
    return 0;
}
```

Output:

```
Integral = 1.357081
```

10. Write a C program to calculate the value of the integral

$$\int_0^1 e^{-x^2} dx$$

by Trapezoidal rule, taking $h = 0.1$

Program:

```
#include <stdio.h>
#include <math.h>

double f(double x) { return exp(-(x*x)); }

int main() {
    double a = 0, b = 1;
    int n = 6;
    double h = (b - a) / n;
    double sum = f(a) + f(b);

    for (int i = 1; i < n; i++)
        sum += 2 * f(a + i * h);

    double result = (h / 2) * sum;
    printf("Integral = %.6f\n", result);
    return 0;
}
```

Output:

```
Integral = 0.745119
```